

ИНЖЕНЕРНЫЙ ПРАКТИКУМ

LLM API с Python

Архитектура, библиотеки и первый программный вызов

Переход от ручного копирования в веб-чате к фоновой автоматизации. Разбираем, как устроена клиент-серверная связка с моделью, зачем нужны клиентские библиотеки и как гарантировать строгие данные на выходе.

Дорожная карта курса: Где мы находимся

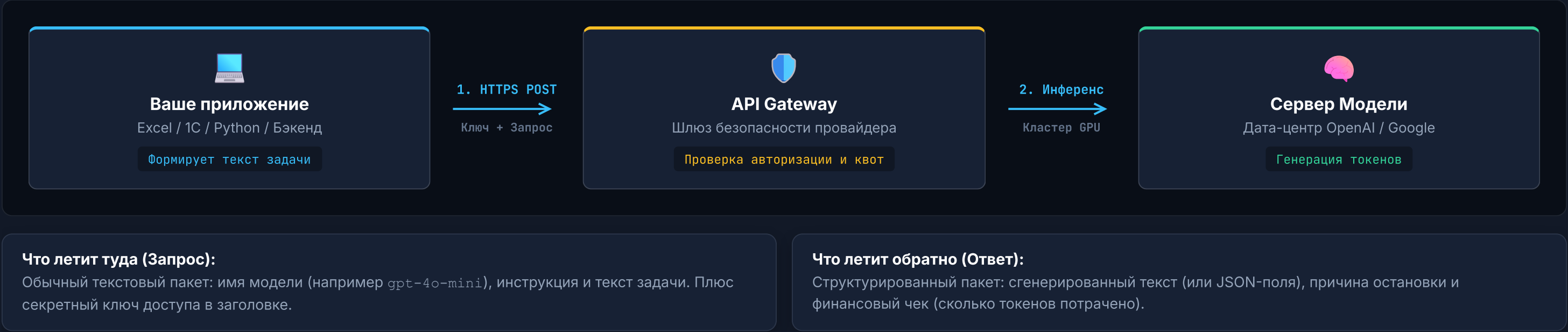
Завершение первого модуля: переход от теории токенов к программному интерфейсу



Смысл сегодняшнего шага: Базовый вызов через API — это фундамент. Без него невозможно отправлять картинки в Модуле 2, подключать функции в Модуле 5 или строить автономных агентов в Модуле 6.

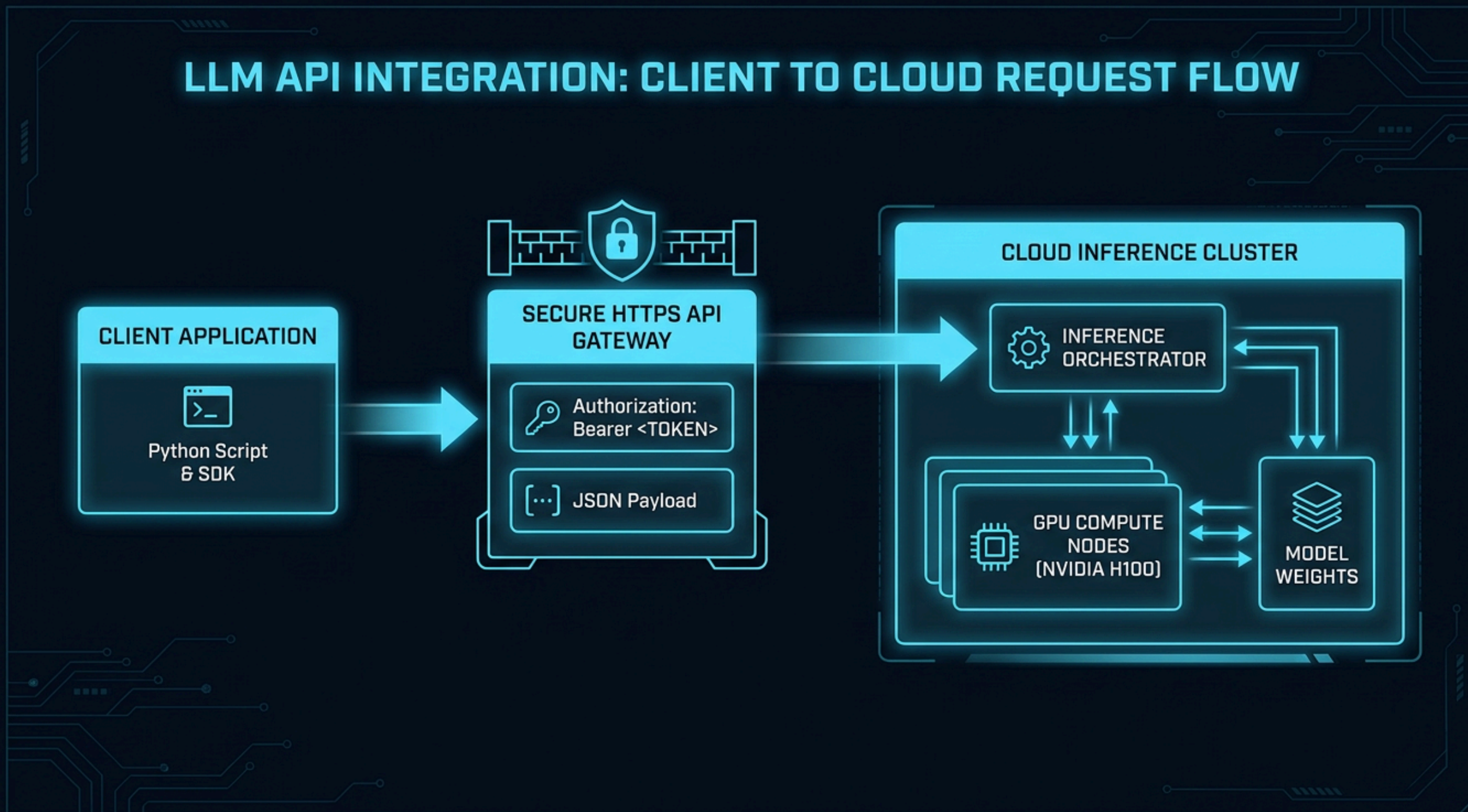
Как софт общается с AI: Архитектура взаимодействия

Модель не живет внутри вашего приложения. Это удаленный сетевой сервис



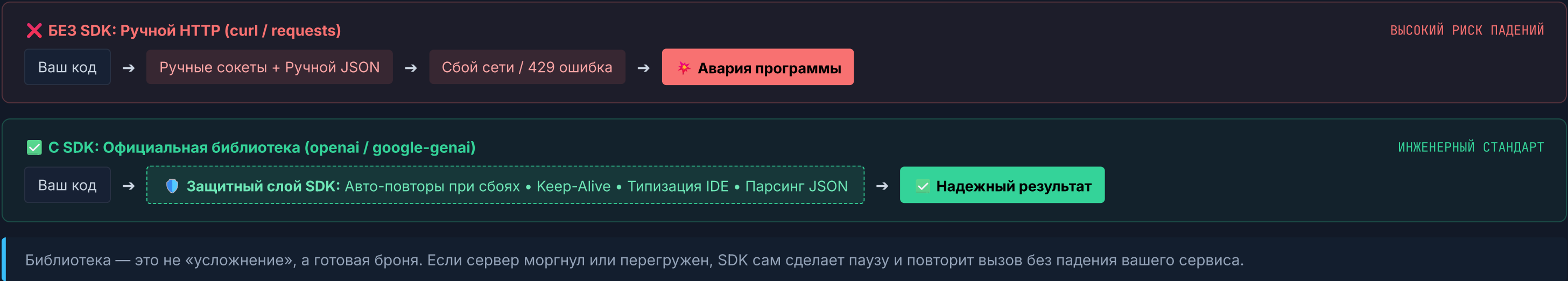
Анатомия HTTP-запроса: Клиент, Шлюз и Кластер

Сквозной путь прохождения сетевого пакета от скрипта через границу REST API к ускорителям инференса



Зачем нужен SDK: Защитный слой между софтом и сетью

Библиотека берет на себя 90% сетевой рутины и спасает приложение от аварийных падений



Первый вызов API: От веб-песочницы к коду в Google Colab

Google AI Studio визуально настраивает модель и дает кнопку «Get Code», а Colab исполняет скрипт без утечки ключей

Ячейка ноутбука Google Colab

Кнопка ▶ Play

```
from google import genai
from google.colab import userdata

# 1. Защита ключа: читаем из Colab Secrets (иконка 🔑 слева)
# Ключ привязан к Google-аккаунту: НЕ виден на экране и НЕ попадает в запись
api_key = userdata.get("GEMINI_API_KEY")
client = genai.Client(api_key=api_key)

# 2. Вызываем флагманскую быструю модель 2026 года
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents="В чем суть API в 2 словах?"
)

# 3. Печатаем ответ напрямую
print(response.text)
```

- ### 1. Google AI Studio (Google Labs)

Песочница `aistudio.google.com`: тестируете промпты в браузере и жмете кнопку «Get Code». Писать код с нуля вручную не требуется!
- ### 2. Безопасность: Colab Secrets 🔑

Ключ хранится в системном сейфе `userdata.get()`. Никаких всплывающих окон `getpass` и нуля риска засветить баланс на стриме или демо.
- ### 3. Модель Gemini 3.5 Flash

Актуальный стандарт скорости и рассуждений (контекст 2M+ токенов, нативная поддержка видео и документов, бесплатный Free Tier).

Что вернул сервер: Разбор электронного чека

Вместе с текстом сервер отдает паспорт транзакции: причину остановки и точный расход токенов

ТЕКСТ ОТВЕТА

Свойство `response.text`

Чистый сгенерированный результат. Библиотека автоматически собирает текстовые блоки и убирает служебные заголовки.

ПРИЧИНА ОСТАНОВКИ

`finish_reason`

Значение `STOP` означает штатное завершение мысли.
Если там `MAX_TOKENS` — сработал лимит длины ответа, а не зависание.

СЧЕТЧИК ТОКЕНОВ

`usage_metadata`

Учет расхода: `prompt_token_count` (вопрос) и `candidates_token_count` (ответ). Позволяет контролировать квоты Free tier и бюджет.

```
# Проверка электронного чека транзакции в Google Colab:
m = response.usage_metadata
print(f"Токены: Вход={m.prompt_token_count}, Выход={m.candidates_token_count}, Всего={m.total_token_count}")
print(f"Статус завершения: {response.candidates[0].finish_reason}")
```

Безопасность API-ключей: Защита от утечек и трат

Ключ от API — это кредитная карта с прямым доступом к корпоративному или личному балансу



Слив на GitHub за 3 секунды

Боты круглосуточно сканируют публичные коммиты. Ключ, случайно вставленный прямо в код файла, перехватывается за секунды и опустошает баланс.



Стандарт: .env и .gitignore

Ключ хранится локально в файле .env. Этот файл обязательно вносится в .gitignore, чтобы Git не отправлял его на сервер.



Colab Secrets (Вкладка 🔑)

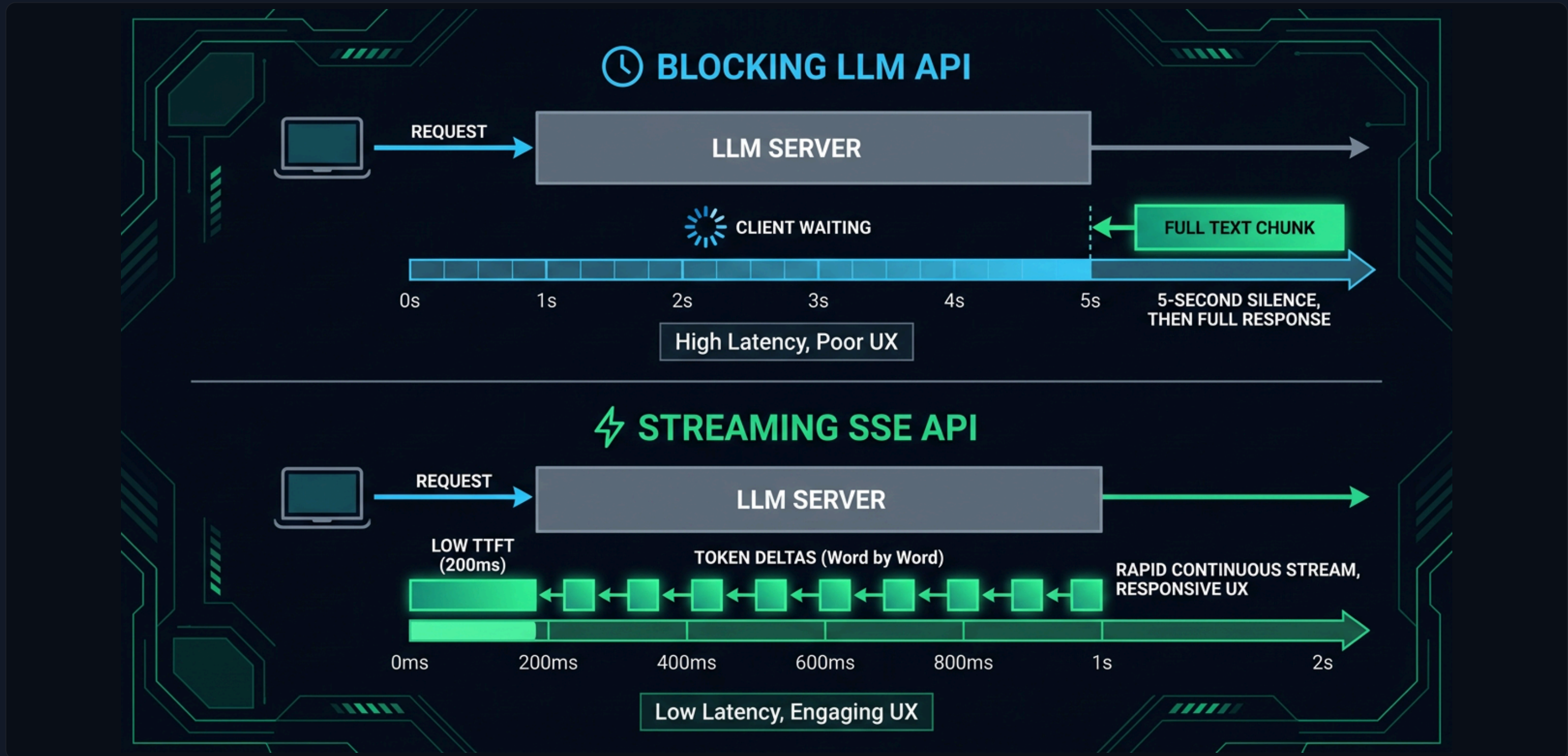
В левом меню Colab нажмите иконку ключа 🔑 → добавьте GEMINI_API_KEY. Ключ привязан к вашему Google-профилю, а не к файлу! При отправке блокнота ключ останется скрыт.

```
# 1. Локально: файл .gitignore блокирует утечку .env в Git:
.env # Защищает от случайного git push в публичный репозиторий

# 2. В Google Colab: чтение из хранилища Secrets (ключ не виден на экране и в записи):
from google.colab import userdata
api_key = userdata.get("GEMINI_API_KEY") # Ключ подтягивается в память без отображения в коде
```

Стриминг: Физика Server-Sent Events и задержка

Сравнение таймлайнов: 5 секунд гнетущей тишины против мгновенного появления первого токена (TTFT)



Стриминг в коде: Разбираем потоковую генерацию

Метод `generate_content_stream` отдает текст порциями по мере вычисления, убирая ожидание

```
# Включаем живой поток токенов (Server-Sent Events):
response = client.models.generate_content_stream(
    model="gemini-3.5-flash",
    contents="Объясни стриминг в 2 предложениях."
)

# Выводим кусочки текста в консоль без буферизации:
for chunk in response:
    print(chunk.text, end="", flush=True)
```

Что такое `chunk`?

Микро-пакет данных из сокета, в котором летит не весь документ, а маленькая дельта — 1 или 2 только что сгенерированных токена.

Зачем `flush=True`?

Принудительно сбрасывает буфер терминала или браузера, заставляя буквы появляться на экране немедленно с эффектом пишущей машинки.

Когда стриминг НЕ нужен?

Для фоновых скриптов, очередей задач и парсинга в JSON стриминг вреден. Используйте синхронный вызов для атомарного ответа.

От чата к коду: Перенос XML и One-Shot из Урока 02 в API

Знания прошлых уроков становятся параметрами SDK: системная инструкция, XML-теги и детерминированность

```
from google.genai import types

# 1. Системная роль вынесена в конфигурацию (защита от взлома промпта)
config = types.GenerateContentConfig(
    system_instruction="Ты строгий аудитор. Извлекай факты без вежливости.",
    temperature=0.0 # 0.0 = максимальный детерминизм
)

# 2. XML-каркас и One-Shot пример из Урока 02 внутри строки:
prompt = f"""
<context>Входящая рабочая рекламация</context>
<rules>Извлеки сумму ущерба и отдел</rules>
<example>
    <input>Списали 500$ по ошибке в бухгалтерии</input>
    <output>отдел: биллинг | сумма: 500$</output>
</example>
<document>{raw_text}</document>
"""

response = client.models.generate_content(
    model="gemini-3.5-flash", contents=prompt, config=config
)
```

1. `system\_instruction` в параметрах

В коде системная роль не смешивается с текстом пользователя. Это физически защищает модель от инъекций (Prompt Injection) со стороны злоумышленников.

2. XML-теги внутри f-строк

Теги <context>, <rules> и <document> четко разделяют инструкции от сырых входящих данных любого объема.

3. One-Shot + `temperature=0.0`

Один эталонный пример в промпте и нулевая температура полностью исключают лишнюю «вежливость» модели и гарантируют одинаковый результат при каждом запуске.

Constrained Decoding: грамматическая маска обнуляет вероятность недопустимых знаков при генерации

1. Foundation Model Input



Token Logits & Probabilities (Unconstrained)

Token Logit. & Probabilities
(Unconstrained)

```
(e.g., {
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "age": {"type": "integer"},
    "skills": {"type": "array",
      "items": {"type": "string"}}
  }
},
"required": ["name", "age",
  "skills"]
))
```

**Filtered Valid
Token Logits**
(e.g., only valid
next-token)

```
{
    {",
    "name",
    P,
    ",
    "
}
```

3. Guaranteed Valid JSON Output

Strict Types

```
{
  "name": "Alice Chen",
  "age": 30,
  "skills": [
    "Python",
    "C++",
    "Machine Learning"
  ]
}
```

Zero Syntax Errors

Schema Compliant

Valid Structure

Практический пример: Разбор входящего письма в JSON

Параметр `response\_mime\_type="application/json"` и температура 0.0 для жесткого контракта данных

```
import json
from google.genai import types

# Жесткий режим: модель физически не может вернуть ничего, кроме JSON
config = types.GenerateContentConfig(
    response_mime_type="application/json",
    temperature=0.0 # Ноль фантазии, строгий детерминизм
)

prompt = """
Разбери входящую заявку в JSON:
{
  "category": "billing" | "technical" | "general",
  "urgent": true | false,
  "amount_usd": float
}
Заявка: Срочно! С карты дважды списали $450 за подписку, верните деньги!
"""

response = client.models.generate_content(
    model="gemini-3.5-flash", contents=prompt, config=config
)

# Безопасно парсим валидный JSON в словарь Python:
data = json.loads(response.text)
print(data["category"]) # → "billing"
print(data["amount_usd"]) # → 450.0
print(data["urgent"]) # → True
```

Что произошло?

Эмоциональное неструктурированное письмо клиента за 1 секунду превратилось в четкую карточку с готовыми данными.

Зачем `temperature=0.0`?

Для извлечения цифр и категорий фантазия модели опасна. Нулевая температура гарантирует максимальную детерминированность.

Куда идут данные?

Поле `amount_usd` сразу идет в учетную систему, а срочная жалоба летит в финотдел без участия ручного оператора.



Экватор занятия: Кофе-пауза

Отдохните 5 минут и переведите дыхание. Во второй части разберем сетевые ошибки, ландшафт библиотек и запустим готовый интерактивный блокнот в Google Colab.

05 : 00

СТАРТ 5 МИН

СБРОС

Что может пойти не так: 4 частые сетевые ошибки

Сеть обрывается, облака сбоят, а квоты исчерпываются. Как софт реагирует на ошибки

401 UNAUTHORIZED

Неверный ключ

Ключ скопирован с лишним пробелом, отозван или не задан в переменных.
Решение: перепроверить значение ключа.

429 RATE LIMIT

Превышен лимит

Слишком частые запросы в минуту или закончился баланс кредитов. Решение: пауза и экспоненциальный откат (Backoff).

500 / 503 ERROR

Сбой провайдера

Перегрузка в дата-центре OpenAI или Google. Решение: 1–2 автоматических повтора или переход на резервную модель.

TIMEOUT

Зависание сети

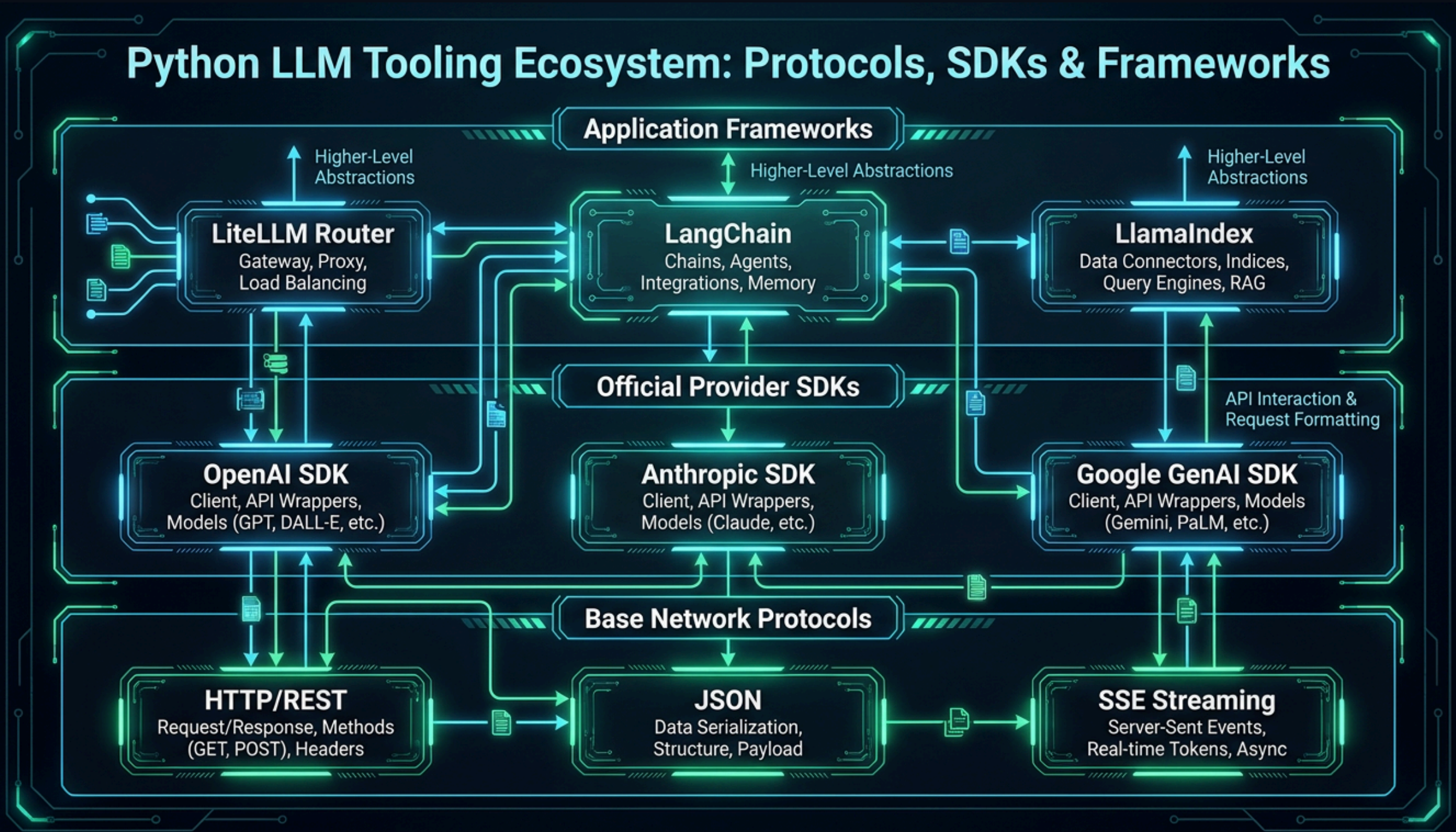
Пакеты застряли по дороге. Без параметра `timeout=30.0` скрипт может зависнуть намертво.



Встроенные ретраи: Официальный SDK уже содержит встроенную логику повторов на ошибки 429 и 500. Вам не нужно писать сложные циклы руками.

Ландшафт библиотек: Кто есть кто в мире Python AI

От базового сетевого уровня до официальных SDK и высокоуровневых фреймворков оркестрации



Сравнение провайдеров: OpenAI vs Claude vs Gemini

Разные имена методов — единая логика. Освоив один SDK, вы умеете работать со всей индустрией

OPENAI SDK (2026)

Стандарт вызова функций

```
client.chat.completions.create(  
  model="gpt-4o", # или o3-mini  
  response_format={...}  
)
```

Отраслевой эталон Structured Outputs и Function Calling.
Поддерживается локальными серверами (vLLM, Ollama).

ANTHROPIC SDK (2026)

Глубокие рассуждения и код

```
client.messages.create(  
  model="claude-3-7-sonnet",  
  thinking={"type": "enabled"}  
)
```

Флагман архитектурного мышления и кодогенерации.
Поддерживает нативный режим расширенного обдумывания (Thinking Mode).

GOOGLE GENAI SDK (2026)

Окно 2M+, видео и Free Tier

```
client.models.generate_content(  
  model="gemini-3.5-flash",  
  contents=[...]  
)
```

Флагманская скорость, нативный анализ видео и аудио,
2M+ токенов контекста и 15 бесплатных вызовов в минуту без привязки карт.

💡 А открытые модели **DeepSeek-V3 / R1** и библиотека **LiteLLM** позволяют переключаться между провайдерами одной строчкой кода.

Чек-лист первого AI-скрипта: 5 правил инженера

Контрольный список перед тем, как отдать скрипт пользователю или запустить в проде

1. Гигиена ключей

ОБЯЗАТЕЛЬНО

Ключи только в `.env` или переменных окружения. Файл `.env` обязательно проверен в `.gitignore`.

2. Эксплицитный таймаут

НАДЕЖНОСТЬ

Всегда задавать `timeout=30.0` при создании клиента, чтобы зависший сокет не парализовал приложение.

3. Температура под задачу

ТОЧНОСТЬ

`temperature=0.0` для JSON, таблиц и классификации. `0.7+` только для креативных текстов.

4. Формат через `response_format`

КОНТРАКТ

Никогда не надеяться на текстовую просьбу «верни JSON». Использовать строгий режим JSON-схемы от API.

5. Логирование телеметрии

КОНТРОЛЬ

Всегда сохранять `response.id` и потраченные `usage.total_tokens` для учета финансов.

Домашнее задание: Запуск в Colab и WOW-кейс с YouTube

Два трека на выбор: базовый триаж документов в JSON или автоматический конвейер анализа видео

ТРЕК А • БАЗОВЫЙ

Триаж рабочего документа в JSON

- 1. Возьмите реальный фрагмент вашей рабочей переписки (письмо, баг, заявка на оплату).
- 2. Запустите готовый блокнот `L03_Google_Colab_Quickstart.ipynb`.
- 3. Получите структурированный `result.json` с обязательными полями (категория, сумма, приоритет).

Фокус: Constrained Decoding & Pydantic

ТРЕК Б • WOW-ЭФФЕКТ (CREATOR TOOLS PATTERN)

YouTube & Meeting Video Analyzer

- 1. Вставьте в ячейку Colab ссылку на любое техническое видео или вебинар с YouTube.
- 2. Скрипт за 1 сек забирает транскрипт, упаковывает в XML-шаблон из Урока 02 и передает в `gemini-3.5-flash`.
- 3. Модель выдает **Executive Memo**: выжимку (TL;DR), ключевые решения с таймкодами и список поручений.

Фокус: Экономия 2 часов просмотра видео

Вопросы и ответы: Главные итоги

Открытый микрофон и анонс второго модуля — переход к изображениям и звуку

3 ГЛАВНЫХ ВЫВОДА ЗАНЯТИЯ

- 1. **LLM — это удаленный веб-сервер.** Модель не скачивается в скрипт. Вы отправляете обычный HTTP-запрос и получаете обратно токены.
- 2. **Стриминг — для живых людей, батч — для систем.** Поточковая выдача нужна только перед глазами пользователя; для баз данных берем полный ответ.
- 3. **Структурированный JSON спасет ваш софт.** Для интеграции всегда фиксируйте схему и выставляйте температуру 0.0.

СЛЕДУЮЩИЙ УРОК • УРОК 04

Генерация и редактирование изображений

Переходим к **Модулю 2: Мультимодальность**.

Разберем диффузионные модели: **DALL-E 3, Midjourney, Stable Diffusion** и **Google Imagen**. Как генерировать графику, схемы и редактировать визуал через код.



Открываем микрофоны и ждем ваших вопросов в чате!